Sistema de Mensajería Cliente-Servidor con XML-RPC y Tkinter

Nicolle Bustamante Hernández
Ingeniería de Sistemas y Computación
Universidad Tecnológica de Pereira
Pereira, Colombia
n.bustamante@utp.edu.co

***Resumen*—En la práctica se construyó un sistema de mensajería privada con XML-RPC para un servidor Ubuntu y tres clientes Windows. El servidor concentró la autenticación, el historial y los archivos; los clientes usaron Tkinter para mostrar un directorio de contactos y conversaciones en ventanas independientes, simulando la experiencia de un cliente de correo moderno. La seguridad combinó PBKDF2-HMAC-SHA256 con sal y tokens de sesión, y la persistencia se resolvió en JSON con escrituras atómicas. Un heartbeat retiró las sesiones de clientes caídos. Las pruebas verificaron mensajes, archivos, entrega y expiración por inactividad entre los tres clientes.**

***Palabras clave*—XML-RPC, Tkinter, sistemas distribuidos, autenticación, PBKDF2, heartbeat, persistencia JSON.**

I. Introducción

La comunicación entre procesos es una condición necesaria para construir sistemas distribuidos. En una aplicación de mensajería, los datos deben moverse entre programas que no comparten memoria y que pueden ejecutarse sobre sistemas operativos distintos. Los sockets permiten esa comunicación, pero obligan a definir manualmente el formato de los mensajes, el orden de los campos y la forma de reportar los errores. RPC, o llamada de procedimiento remoto, propone una abstracción diferente: el cliente invoca una función del servidor con argumentos y recibe un valor de retorno, mientras la biblioteca se encarga de serializar la llamada y transportarla por la red [1].

Esta abstracción permite abordar la comunicación distribuida desde una perspectiva más cercana a la programación convencional, sin eliminar las dificultades propias de la interacción a través de una red. A diferencia de una llamada local, una invocación remota depende de la disponibilidad del servidor, de la conectividad entre los procesos y de la correcta interpretación de los datos intercambiados. Por ello, el diseño de una aplicación de este tipo requiere considerar no solo la ejecución de las operaciones, sino también la autenticación de los usuarios, el manejo de errores y la conservación del estado necesario para mantener la coherencia de las conversaciones.

En este contexto, una aplicación de mensajería constituye un escenario pertinente para estudiar los mecanismos de comunicación entre procesos, ya que reúne operaciones de intercambio de información, administración de sesiones y transferencia de datos entre usuarios. Su desarrollo permite examinar la relación entre la interfaz del cliente, los servicios ofrecidos por el servidor y el protocolo utilizado para comunicar ambos extremos. Asimismo, plantea cuestiones relacionadas con la concurrencia y la tolerancia a fallos, aspectos fundamentales para comprender el comportamiento de los sistemas distribuidos.

II. Arquitectura y Funcionamiento (RPC)

La aplicación se organizó siguiendo el modelo cliente-servidor. El servidor se encargó de gestionar la base de datos y comprobar que las operaciones solicitadas por los clientes fueran válidas. Por su parte, los clientes enviaron solicitudes al servidor y mostraron las respuestas recibidas. Para atender varias solicitudes, se utilizó `SimpleXMLRPCServer` junto con `ThreadingMixIn`, lo que permitió procesarlas en hilos diferentes. El servidor ofreció nueve operaciones para registrar usuarios, iniciar y cerrar sesiones, comprobar la conexión, consultar los contactos, enviar mensajes y archivos, consultar el historial y borrar conversaciones. La comunicación entre los clientes y el servidor se realizó mediante XML-RPC, que se encargó de convertir los datos al formato necesario para enviarlos por la red y devolver las respuestas a los clientes [2], [3].

En el cliente, la comunicación con el servidor se realizó mediante una función llamada `rpc`, que facilita el envío de solicitudes utilizando la biblioteca XML-RPC de Python. Por ejemplo, cuando un usuario inicia sesión, el cliente envía su nombre de usuario y contraseña al servidor, que comprueba los datos y devuelve una respuesta. Aunque estas llamadas se escriben de forma similar a una función normal de Python, dependen de la conexión de red y de que el servidor esté disponible. Por esta razón, el cliente contempla situaciones como la falta de conexión, los tiempos de espera agotados y los errores devueltos por el servidor, y muestra mensajes para informar al usuario cuando una operación no puede completarse.

```python
def rpc(metodo, *argumentos):
    with xmlrpc.client.ServerProxy(
        URL_SERVIDOR,
        allow_none=True,
        transport=TransporteConTiempo(),
    ) as proxy:
        return getattr(proxy, metodo)(*argumentos)
```

Listing 1. Invocación remota desde el cliente mediante `ServerProxy`.

La autenticación se gestionó mediante un token que el servidor generó cuando un usuario iniciaba sesión correctamente. Para crearlo, se utilizó `secrets.token_urlsafe(48)`, y el token se guardó en un diccionario en memoria asociado al nombre del usuario. A partir de ese momento, el cliente debía enviar el token en las solicitudes que requerían autenticación, de modo que el servidor pudiera comprobar quién realizaba cada operación. Esta verificación se hizo mediante la función `_autenticar`, que también actualizaba la última actividad del usuario para controlar si la sesión seguía activa.

Si una solicitud incluía un token inválido, el servidor devolvía un error XML-RPC con el código 401, y el cliente regresaba a la pantalla de inicio de sesión. Los tokens se mantuvieron únicamente en la memoria del servidor y no se guardaron en el archivo JSON utilizado para almacenar los datos de la aplicación. Por esta razón, al reiniciar el servidor, las sesiones existentes dejaban de ser válidas y los usuarios debían iniciar sesión nuevamente.

```python
def _autenticar(token):
    if not isinstance(token, str) or not token:
        raise Fault(401, "Sesión inválida. Inicia sesión
     nuevamente.")
    with _candado:
        nickname = sesiones.get(token)
        if nickname:
            ultimos_latidos[token] = time.time()
    if not nickname:
        raise Fault(401, "Sesión inválida o expirada. Inicia sesi
     ón nuevamente.")
    return nickname


def revisar_sesiones_inactivas():
    ahora = time.time()
    expirados = []
    with _candado:
        for token in list(sesiones):
            ultimo = ultimos_latidos.get(token)
            if ultimo is None or ahora - ultimo > TIMEOUT_LATIDO:
                nickname = sesiones.pop(token, None)
                ultimos_latidos.pop(token, None)
                if nickname:
                    expirados.append(nickname)
    for nickname in expirados:
        print(f"[SISTEMA] Usuario {nickname} desconectado por
     inactividad.")


def _hilo_limpieza():
    while not _parar_limpieza.wait(INTERVALO_LIMPIEZA):
        revisar_sesiones_inactivas()
```

Listing 2. Validación del token y actualización del heartbeat en el servidor.

La autenticación de usuarios y la autenticación de sesiones fueron mecanismos distintos. La primera se resolvió en el registro y en el `login`, donde se comprobó la contraseña; la segunda verificó el token en cada operación. Esa separación evitó enviar la contraseña de forma repetida y permitió que una sesión se cerrara sin afectar a las demás. También explicó por qué tres clientes en el mismo Windows pudieron convivir sin confundirse: el servidor no los identificó por IP, sino por el token que cada proceso conservaba en memoria.

III. Interfaz Gráfica (Tkinter)

La interfaz gráfica se desarrolló con Tkinter y `ttk` para permitir la interacción con los servicios ofrecidos por el servidor XML-RPC. Desde el cliente es posible iniciar sesión, consultar el directorio de usuarios, intercambiar mensajes y enviar archivos. La interfaz actúa como intermediaria entre el usuario y las operaciones remotas, mostrando los resultados recibidos y los errores que puedan producirse durante la comunicación [4].

Para evitar que las llamadas RPC bloquearan la interfaz mientras esperaban una respuesta, las solicitudes se ejecutaron en hilos de trabajo independientes. Los resultados se depositaron en una cola y se procesaron desde el hilo principal mediante `after()`, encargado de actualizar los componentes gráficos. Esta separación permitió mantener la interfaz receptiva durante las operaciones de red y evitó modificar los componentes de Tkinter desde hilos secundarios.

Como XML-RPC no proporciona notificaciones automáticas del servidor hacia los clientes, la actualización del directorio y de los historiales se realizó mediante consultas periódicas. El directorio se actualizó cada 4 segundos y el historial de cada conversación abierta cada 2,2 segundos. Este mecanismo permitió detectar nuevos mensajes y cambios en la información de los contactos mediante solicitudes al servidor, a costa de generar consultas recurrentes durante la ejecución de la aplicación. Para simular las tres máquinas clientes solicitadas, se ejecutaron tres instancias del cliente en el equipo anfitrión (Windows), cada una representando a un usuario distinto (`cliente1, cliente2 y cliente3`).



Figura 1. Pantalla de registro e inicio de sesión del cliente.

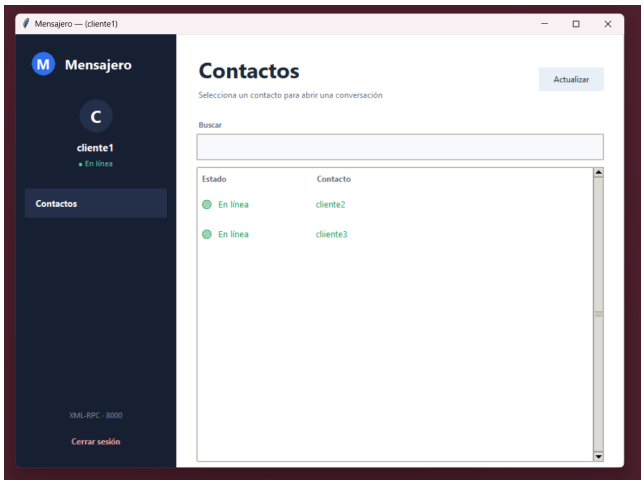


Figura 2. Ventana principal con el directorio de contactos, la búsqueda y el estado de conexión.

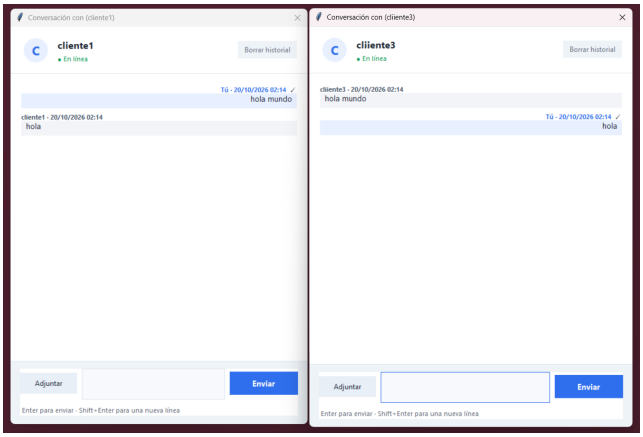


Figura 3. Dos conversaciones abiertas en ventanas `Toplevel` independientes.

IV. Transferencia de archivos y persistencia

El envío de archivos se realizó mediante `xmlrpc.client.Binary`, que permite transmitir datos binarios a través de XML-RPC. Antes del envío, el cliente leyó el archivo en modo binario y comprobó que su tamaño no superara los 5 MB. Después, la biblioteca convirtió su contenido a Base64 para incluirlo en el mensaje XML. Este procedimiento permitió enviar archivos de distintos tipos, como imágenes, documentos y sonidos, sin necesidad de interpretar su contenido. El servidor recibió los bytes originales, los codificó en Base64 para almacenarlos en el archivo JSON y los devolvió al cliente cuando este consultó el historial. Finalmente, el destinatario pudo elegir dónde guardar el archivo recibido. El tamaño se verificó tanto en el cliente como en el servidor, y el nombre del archivo se validó para evitar que incluyera rutas de acceso no permitidas.

```
def tarea():
    with open(ruta, "rb") as archivo:
        contenido = archivo.read(MAX_ARCHIVO_BYTES + 1)
    if len(contenido) > MAX_ARCHIVO_BYTES:
        raise ValueError("El archivo supera el límite de 5 MB.")
    return rpc(
        "enviar_archivo",
        self.app.token,
        self.contacto,
        nombre,
        xmlrpc.client.Binary(contenido),
    )
```

Listing 3. Lectura y envío de un archivo como `xmlrpc.client.Binary`.

La información de la aplicación se almacenó en `datos_servidor.json`, que contiene los usuarios registrados y los mensajes intercambiados. En cada mensaje se guardaron datos como el identificador, el emisor, el destinatario, la fecha y el tipo de contenido. Para los archivos adjuntos también se almacenaron el nombre, el tamaño y el contenido codificado en Base64. Para reducir el riesgo de perder información si el proceso de escritura se interrumpía, el servidor guardó los cambios en un archivo temporal y luego lo utilizó para reemplazar el archivo original mediante `os.replace`. Además, al borrar una

conversación, se eliminaron los mensajes intercambiados en ambos sentidos y se guardaron los cambios, de modo que el historial desapareciera para los dos participantes.

IV-A. Seguridad y autenticación

Las contraseñas no se almacenaron directamente en el archivo JSON. En su lugar, el servidor utilizó PBKDF2-HMAC-SHA256 con 310,000 iteraciones para obtener un hash a partir de cada contraseña y una sal aleatoria de 16 bytes [5]. Durante el inicio de sesión, el servidor repitió el cálculo y comparó el resultado con el valor almacenado mediante `hmac.compare_digest`. De esta forma, pudo comprobar la contraseña sin guardar su valor original. También se realizó un cálculo ficticio cuando el nickname no existía, con el propósito de reducir las diferencias en el tiempo de respuesta que pudieran revelar si una cuenta estaba registrada.

Una vez autenticado el usuario, el servidor generó un token de sesión mediante `secrets`. Este token se utilizó para identificar al usuario en las operaciones que requerían autenticación y se eliminó al cerrar sesión o cuando se detectó inactividad. Asimismo, se validaron los datos recibidos, incluidos los nicknames, las contraseñas, los mensajes y los nombres de los archivos, antes de procesar las solicitudes.

Finalmente, la comunicación se realizó mediante XML-RPC sobre HTTP, sin utilizar TLS. Aunque la red *host-only* limitó el alcance de la comunicación al entorno del laboratorio, esto no significa que los datos viajaran cifrados. Los hashes protegen las contraseñas almacenadas y los tokens permiten controlar el acceso a las operaciones, pero ninguno de estos mecanismos cifra los mensajes transmitidos por la red. Para utilizar la aplicación en un entorno real, sería necesario proteger el canal mediante HTTPS.

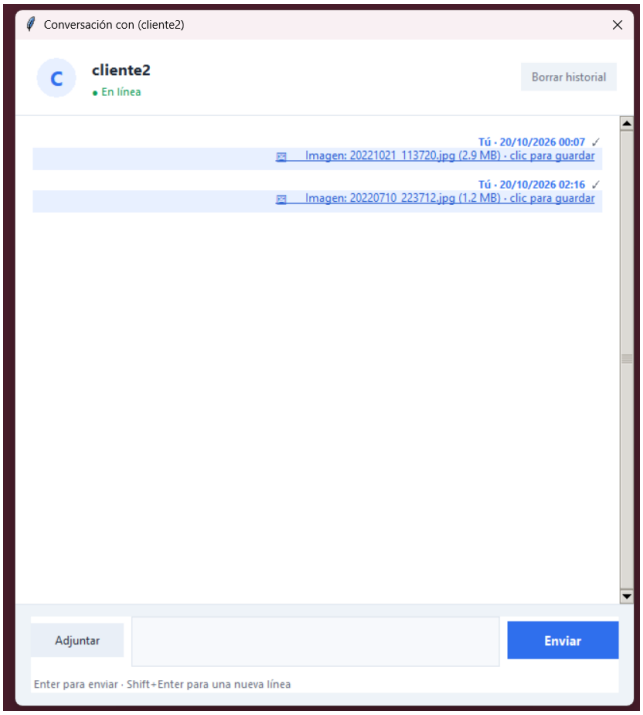


Figura 4. Transferencia de un archivo y vista del adjunto con su tipo, tamaño y opción de guardado.

V. Diagrama de Funcionamiento

La Figura 5 resume el flujo completo. En la autenticación, cada cliente se registra, inicia sesión y recibe un token. Después consulta el directorio para conocer los usuarios registrados y su estado de conexión. En la mensajería, el emisor invoca `enviar_mensaje`, el servidor persiste el mensaje y el receptor lo obtiene al consultar su historial. En la transferencia, el archivo viaja como `Binary`, se guarda en Base64 y se reconstruye cuando el destinatario lo consulta. El diagrama incluye el mecanismo de *heartbeat*, mediante el cual los clientes envían periódicamente la operación `ping` para actualizar su actividad. Si una sesión supera el tiempo de inactividad establecido, el servidor la elimina.

VI. Dificultades Encontradas

Una de las principales dificultades fue ejecutar la interfaz gráfica en la máquina virtual cliente. El entorno de Ubuntu Server no disponía de un entorno gráfico configurado para abrir ventanas de Tkinter. Por esta razón, se utilizó el equipo anfitrión con Windows para ejecutar los clientes, mientras que la máquina virtual se destinó a ejecutar el servidor RPC. Para cumplir con los 3 clientes solicitados, se ejecutaron tres instancias del cliente de Windows, cada una simulando una máquina independiente conectada al servidor en la VM.

Otra dificultad fue evitar que las llamadas RPC bloquearan la interfaz. Como las solicitudes esperan una respuesta del servidor, ejecutarlas directamente en el hilo principal podía congelar la ventana. Para solucionarlo, las llamadas

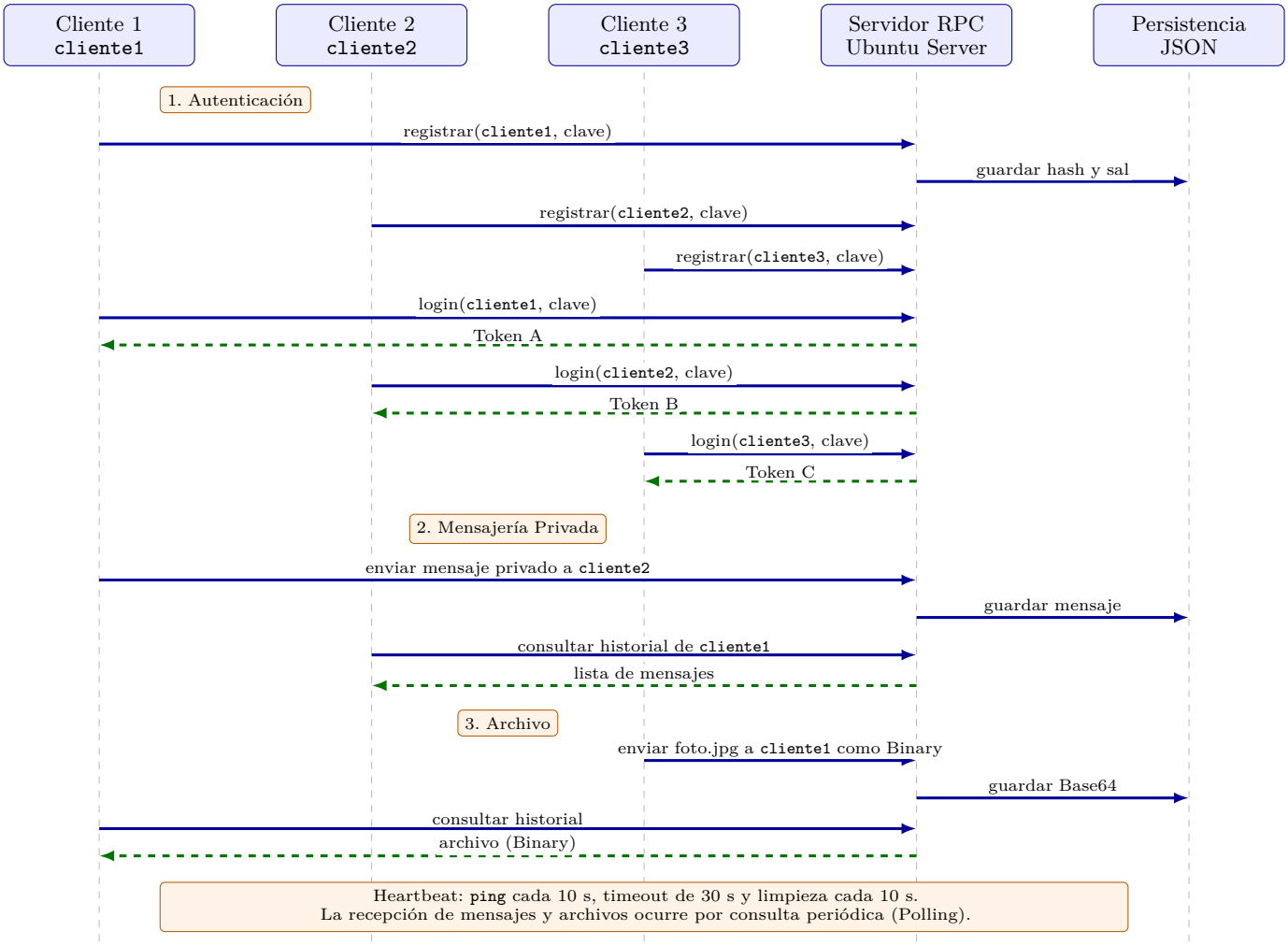


Figura 5. Diagrama de secuencia del registro, autenticación, mensajería privada, transferencia de archivos y heartbeat para los tres clientes.

se ejecutaron en hilos secundarios y sus resultados se procesaron mediante una cola de eventos.

También fue necesario controlar las sesiones de usuarios que se desconectaban sin cerrar sesión. Para ello, se implementó un mecanismo de *heartbeat* mediante la operación `ping`, que permite actualizar la actividad de cada sesión y eliminar aquellas que superan los 30 segundos de inactividad.

Finalmente, se controló el acceso concurrente a los datos compartidos del servidor mediante un `RLock`. Además, las escrituras del archivo JSON se realizaron de forma atómica para reducir el riesgo de corrupción de los datos si varias solicitudes modificaban el estado del sistema.

```
socketlab@VM-SERVER:~$ python3 servidor_rpc.py
[DATOS] Cargados 3 usuario(s) y 4 mensaje(s).
============================================================
Servidor XML-RPC escuchando en 0.0.0.0:8000
Clientes Windows: http://192.168.56.10:8000
Archivo de datos: /home/socketlab/datos_servidor.json
Ctrl+C para detener el servidor.
============================================================
[LOGIN] cliente1 conectado.
[LOGIN] cliente2 conectado.
[LOGIN] cliiente3 conectado.
[SISTEMA] Usuario cliiente3 desconectado por inactividad.
```

Figura 6. Consola del servidor durante la expiración de una sesión por inactividad.

VII. Conclusiones

La práctica permitió comprender cómo RPC simplifica la comunicación entre procesos al permitir que un cliente invoque operaciones de un servidor sin implementar manualmente el intercambio de mensajes. Sin embargo, la abstracción no elimina los problemas de red, por lo que es necesario controlar los errores, las sesiones y el acceso concurrente a los datos.

También se evidenció que el diseño de un sistema distribuido requiere considerar las limitaciones de las

decisiones adoptadas. En este caso, el almacenamiento en JSON facilitó la persistencia, pero el uso de Base64 para los archivos aumenta el consumo de espacio y la ausencia de TLS deja la comunicación expuesta.

Referencias

[1] A. S. Tanenbaum y M. van Steen, *Distributed Systems*, 4.ª ed. Maarten van Steen, 2017.

[2] Python Software Foundation, "xmlrpc — XML-RPC client and server modules," *Python 3 Documentation*. [En línea]. Disponible en: https://docs.python.org/3/library/xmlrpc.html

[3] D. Winer, "XML-RPC Specification," 1999. [En línea]. Disponible en: https://www.xmlrpc.com/spec

[4] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," *Python 3 Documentation*. [En línea]. Disponible en: https://docs.python.org/3/library/tkinter.html

[5] Python Software Foundation, "hashlib — Secure hashes and message digests," *Python 3 Documentation*. [En línea]. Disponible en: https://docs.python.org/3/library/hashlib.html

[6] G. Coulouris, J. Dollimore, T. Kindberg y G. Blair, *Distributed Systems: Concepts and Design*, 5.ª ed. Addison-Wesley, 2011.